point in the margin regions is a core point, obviously[19] we need to merge the two clusters (or, equivalently, get rid of C_6).

Likewise, if a point `p` is classified as core point in one shard and noise in the other, there won't be clusters to merge: `p` will already be in the right cluster, but we also need to be careful, because it should be removed from the list of outliers.

The output of each reducer will be a list of local clusters to merge or, alternatively, the disjoint set keeping track of the merges. A further, brief, composition step can take care of producing the list with the points assignments to the final clusters and the list of outliers, based on reducers' output.

The pseudo code for this MapReduce job is simpler than any other job in this chapter, but don't be fooled: most of the complexity is hidden in the methods `gridShard`, `dbscan` and `mergeClusters`.

Method `dbscan` is exactly the same method that we have described in section 12.3: in most languages we will be able to re-use it without any modification; `gridShard`, in its most naïve version, just iterates over points and computes the index of the cell by performing a modulo division (plus a few checks to see if the point is in the margin of adjacent cells) - we'll address some problematics connected to this method later in this section, but we won't get into the details of its implementation.

Finally, method `mergeClusters`: this is a nice application of disjoint set, the data structure we described in chapter 5.